
Volante VINIK + Unity

Guia completo: WheelColliders, Input e Configuração

Este guia explica, passo a passo, como os scripts **WheelAxisDebugger.cs** e **CarController.cs** funcionam e como configurar o projeto na Unity para que o volante e os pedais físicos VINIK controlem o carro usando WheelColliders.

Escopo: leitura de eixos analógicos via Input Manager, direção (steerAngle), aceleração/frenagem/ré via motorTorque e brakeTorque, e sincronização visual das rodas.

Sumário

- 1. Visão geral do projeto
- 2. Script 1 — WheelAxisDebugger.cs (descoberta de eixos)
- 3. Script 2 — CarController.cs (controle do carro)
- 4. Configuração passo a passo na Unity
- 5. Problemas comuns e soluções

1. Visão geral do projeto

O objetivo é ligar um volante físico USB (VINIK) a um carro simulado com física real na Unity, usando **WheelCollider** — o componente nativo da Unity para simulação de rodas com suspensão, atrito e torque.

O fluxo geral é:

- O volante/pedais enviam sinais analógicos (eixos) para o sistema operacional.
- A Unity lê esses eixos através do **Input Manager** (Edit > Project Settings > Input Manager).
- O **WheelAxisDebugger** mostra no Console qual eixo corresponde a cada controle físico.
- O **CarController** usa esses eixos para mover os WheelColliders do carro (direção, aceleração, freio e ré).

Por que dois scripts separados?

O WheelAxisDebugger é uma ferramenta de diagnóstico, usada uma única vez (ou sempre que trocar de volante) para descobrir os nomes dos eixos. O CarController é o script definitivo, que fica no carro durante o jogo todo.

2. Script 1 — WheelAxisDebugger.cs

Esse script não controla o carro: ele serve apenas para descobrir quais eixos ("Joy1 Axis 1", "Joy1 Axis 2"...) o volante VINIK está usando, antes de configurar o script principal.

2.1 Detecção do dispositivo

No **Start()**, o script chama **Input.GetJoystickNames()**, que retorna a lista de dispositivos joystick/volante conectados. Se essa lista vier vazia, o problema é de reconhecimento do hardware pelo Windows, não do script.

```
string[] dispositivosConectados = Input.GetJoystickNames();
Debug.Log("Dispositivos detectados: " + dispositivosConectados.Length);
```

2.2 Varredura dos eixos

No **Update()**, um laço percorre "Joy1 Axis 1" até "Joy1 Axis N" e lê cada um com **Input.GetAxisRaw()**. Usa-se a versão "Raw" (sem suavização) porque, para diagnóstico, queremos o valor exato que o volante está enviando, sem interpolação da Unity.

```
string nomeDoEixo = "Joy1 Axis " + i;
float valorAtual = Input.GetAxisRaw(nomeDoEixo);
```

O bloco **try/catch** existe porque tentar ler um eixo que não foi criado no Input Manager lançaria um erro na Unity — o catch simplesmente ignora e segue para o próximo eixo.

2.3 Log apenas quando o valor muda

Sem esse filtro, o Console imprimiria dezenas de linhas por segundo para cada eixo, mesmo parado. O script só registra uma linha quando a diferença em relação à última leitura ultrapassa o **limiarDeAtividade** (padrão: 0.05). Assim, ao girar o volante, só aparece a linha do eixo que está realmente reagindo — esse é o eixo procurado.

```
float diferenca = Mathf.Abs(valorAtual - ultimosValores[i - 1]);
if (diferenca > limiarDeAtividade)
{
    Debug.Log(nomeDoEixo + " = " + valorAtual.ToString("F3"));
}
```

2.4 Detecção de botões

Algumas pedaleiras mais simples enviam o pedal como **botão digital** (0 ou 1) em vez de eixo analógico. Por isso o script também testa os botões 0 a 19 do dispositivo e registra qual foi pressionado.

```
string nomeDoBotao = "joystick 1 button " + botao;
if (Input.GetKeyDown(nomeDoBotao))
    Debug.Log("Botão pressionado: " + nomeDoBotao);
```

3. Script 2 — CarController.cs

Esse é o script definitivo, responsável por mover o carro de acordo com o volante e os pedais, usando WheelColliders.

3.1 Campos públicos configuráveis

Os nomes dos eixos, as referências aos 4 WheelColliders, os ângulos e torques, e duas opções de inversão (**inverterEixoVolante** e **inverterEixoPedais**) ficam expostos no Inspector, porque cada volante mapeia esses sinais de um jeito diferente — é mais prático ajustar visualmente do que reescrever código toda vez.

3.2 Update() x FixedUpdate()

A leitura do input acontece no **Update()** (a cada frame renderizado, garantindo resposta imediata ao volante). Já a aplicação de física (direção, torque, freio) acontece no **FixedUpdate()**, porque WheelCollider é um componente de física — mexer nele fora do passo fixo de simulação causa comportamento instável, independente do framerate.

```
void Update() { LerInputsDoVolante(); }  
void FixedUpdate() { AplicarDirecao(); AplicarAceleracaoEFreio(); AtualizarMalhasVisuais(); }
```

3.3 Leitura e conversão dos pedais

Aqui usa-se **Input.GetAxis()** (com suavização), diferente do debugger — a interpolação da Unity deixa a direção mais fluida durante o jogo. Muitos volantes reportam o pedal solto como -1 e o pedal fundo como 1; o método abaixo converte esse intervalo para 0 (solto) até 1 (fundo), que é mais intuitivo para o resto da lógica.

```
float ConverterPedalParaIntervalo0a1(float valorBruto)  
{  
    float valorConvertido = (valorBruto + 1f) / 2f;  
    return Mathf.Clamp01(valorConvertido);  
}
```

Atenção

Se o seu VINIK já enviar os pedais no intervalo 0 a 1 nativamente, essa conversão vai distorcer os valores. Use o WheelAxisDebugger para conferir e, se precisar, marque a opção "Inverter Eixo Pedais" ou ajuste a fórmula.

3.4 Direção (esterço)

O valor do volante (-1 a 1) é multiplicado pelo ângulo máximo de esterço e aplicado diretamente na propriedade **steerAngle** das duas rodas dianteiras — é essa propriedade que faz o WheelCollider recalculer o contato físico com o chão em ângulo.

```
float anguloDeEsterco = valorVolante * anguloMaximoDeEsterco;  
rodaDianteiraEsquerda.steerAngle = anguloDeEsterco;  
rodaDianteiraDireita.steerAngle = anguloDeEsterco;
```

3.5 Aceleração, freio e ré

Usa-se um **limiar** (padrão 0.05) em vez de comparar com zero exato, porque sensores analógicos raramente ficam em 0.000 — sem esse limiar o carro ficaria "vibrando" o torque o tempo todo. A partir

daí, a lógica segue três casos:

Só acelerador pressionado	Torque positivo nas rodas traseiras → carro anda para frente
Só freio pressionado	brakeTorque aplicado nas 4 rodas → carro freia
Acelerador + freio juntos	Torque negativo nas rodas traseiras → carro dá ré

O motor aplica torque somente nas rodas **traseiras** (tração traseira). O freio, por outro lado, é aplicado nas **4 rodas**, como em um carro real, ajudando a manter o controle ao frear forte.

```
rodaTraseiraEsquerda.motorTorque = torqueAAplicar;  
rodaTraseiraDireita.motorTorque = torqueAAplicar;  
  
rodaDianteiraEsquerda.brakeTorque = freioAAplicar;  
rodaDianteiraDireita.brakeTorque = freioAAplicar;  
rodaTraseiraEsquerda.brakeTorque = freioAAplicar;  
rodaTraseiraDireita.brakeTorque = freioAAplicar;
```

3.6 Sincronização visual das rodas

O WheelCollider é **invisível** — ele só calcula física, não gira nenhuma malha 3D. O método **GetWorldPose()** retorna a posição e rotação reais calculadas pela física (incluindo suspensão e esterço), que são então aplicadas na malha visual da roda. Se nenhuma malha for configurada, o carro funciona igual — só não gira visualmente.

```
Vector3 posicao;  
Quaternion rotacao;  
colisorDaRoda.GetWorldPose(out posicao, out rotacao);  
  
malhaDaRoda.position = posicao;  
malhaDaRoda.rotation = rotacao;
```

4. Configuração passo a passo na Unity

Passo 1 — Verificar o volante no Windows

Painel de Controle > Dispositivos e Impressoras > Configurações de jogo. Gire o volante e pise nos pedais; se as barras se mexerem, o dispositivo está reconhecido e pronto para a Unity.

Passo 2 — Criar os eixos no Input Manager

Edit > Project Settings > Input Manager > Axes. Crie 3 entradas novas: "Joy1 Axis 1" (X axis), "Joy1 Axis 2" (Y axis) e "Joy1 Axis 3" (3rd axis), todas do tipo Joystick Axis, com Sensitivity 1 e Dead 0.001.

Passo 3 — Rodar o WheelAxisDebugger

Crie um GameObject vazio, adicione o script, dê Play e abra o Console. Gire o volante e pise nos pedais, anotando qual "Joy1 Axis N" reage a cada ação.

Passo 4 — Montar o carro

Crie um GameObject "Carro" com um Rigidbody (massa em torno de 1200 kg) e adicione o modelo 3D (ou um Cube, para testes) como filho.

Passo 5 — Criar os 4 WheelColliders

Crie 4 filhos vazios posicionados no lugar exato de cada roda e adicione um WheelCollider em cada um, ajustando Radius e Suspension Distance conforme o tamanho real das rodas.

Passo 6 — Configurar o CarController

Adicione o script ao "Carro", preencha os 3 campos de eixo com o que foi descoberto no Passo 3, e arraste os 4 WheelColliders e (opcionalmente) as malhas visuais nos campos correspondentes.

Passo 7 — Testar e ajustar

Dê Play e teste volante, acelerador, freio e a combinação dos dois (ré). Use as opções "Inverter Eixo Volante" e "Inverter Eixo Pedais" se algo estiver ao contrário.

5. Problemas comuns e soluções

Carro não se move mesmo com acelerador fundo

Confira, com o WheelAxisDebugger, se o valor do eixo do acelerador realmente ultrapassa 0.05. Também verifique se o pedal não está mapeado como botão em vez de eixo (o debugger também mostra isso).

Carro derrapa ou desliza de forma estranha

Ajuste o Forward Friction e o Sideways Friction de cada WheelCollider — valores de Stiffness entre 1 e 2 costumam funcionar bem para a maioria dos casos.

Carro capota facilmente nas curvas

Baixe o centro de massa do Rigidbody via script (`rb.centerOfMass = new Vector3(0, -0.5f, 0);`) ou reduza o Suspension Distance.

Volante "treme" ou o carro vira sozinho parado

Aumente o valor de Dead (zona morta) desse eixo no Input Manager, por exemplo para 0.02.

Volante ou pedais invertidos

Marque "Inverter Eixo Volante" e/ou "Inverter Eixo Pedais" no Inspector do CarController — não é necessário editar código.

Guia gerado para o projeto de carro na Unity com volante e pedais VINIK. Scripts de referência: WheelAxisDebugger.cs e CarController.cs.